Artificial Intelligence compact cheatsheet

STEP 1

Core AI · Intelligence, agents and problems

1.1 What AI is and intelligent agents

Definition AI builds agents that perceive and act. This course uses the acting rationally view: choose the action with the best expected performance.

PEAS Describe any task by Performance measure, Environment, Actuators, Sensors.

Environment types Fully or partially observable, deterministic or stochastic, episodic or sequential, static or dynamic, discrete or continuous, single or multi agent.

Agent programs Simple reflex, model based, goal based, utility based, learning. Table driven needs $\sum |P|^t$ rows, so it is impossible.

Rational, not perfect Rationality depends on the performance measure, prior knowledge, available actions and percepts so far. Bad luck is not irrational.

Social machine Needs etiquette, intuitive psychology, common sense, intuitive physics and layers from instinctive to self conscious (Minsky, Society of Mind).

Agent function $f : P^* \to A$

Rational agent
choose $a^* = \text{argmax}_a$ E[performance | percepts so far, a]

Expected performance of an action
$E[U \mid a] = \sum_{s'} P(s' \mid a)\, U(s')$

Size of a table driven agent
$\text{rows} = \sum_{t=1}^{T} |P|^t = |P|(|P|^T - 1)/(|P| - 1)$

TRAP A table driven agent stores one action for every possible percept sequence. With 4 percepts and a lifetime of T steps it needs $4 + 4^2 + \ldots + 4^T$ rows. For a taxi camera this number is larger than the atoms in the universe. Agent programs exist to produce good behaviour from a small program instead of a huge table.

EXAM A rational agent is not one that is always right. It is one that chooses the best action given what it knows. Bad luck does not make a choice irrational.

1.2 Problems as search

Search problem State space, actions with costs C(x, a, y) ≥ ε > 0, initial state, goal test (a condition). Heuristics are optional.

Node STATE, PARENT, ACTION, PATH-COST g(n). Follow PARENT links to rebuild the path.

Tree against graph search Tree search can loop on cycles. Graph search keeps an explored set and skips seen states.

Four properties Completeness, optimality, time and space, measured with b, d and m. Full tree: $(b^{d+1} - 1)/(b - 1)$ nodes.

Water jugs 14 of 20 states reachable from (0, 0). (1, 2) unreachable. Getting 2 gallons in the 4 gallon jug takes 6 actions.

Formulation Search cannot choose the states for you. A good abstraction can shrink the space enormously (8 queens: 1.8×10^{14} to 2,057).

Successor function
$S(x) = \{\langle y, a\rangle : \text{action } a \text{ takes } x \text{ to } y\}$

Path cost $g(n) = \sum_i C(s_i, a_i, s_{i+1})$

Nodes in a full tree
$1 + b + b^2 + \ldots + b^d = (b^{d+1} - 1)/(b - 1) = O(b^d)$

8 puzzle reachable states $9!/2 = 181{,}440$

Paths are much bigger than states
tree nodes up to depth d ≈ b^d while states may be few

TRAP Cycles can cause non termination. Arad → Sibiu → Arad → Sibiu ... is a valid path in tree search, so a search may never stop. Graph search fixes this by remembering explored states.

EXAM Exam questions often ask you to formulate a problem. Always list all four parts: states, actions (with costs), initial state, goal test. Say how big the state space is.

STEP 2 Core AI · Uninformed search

2.1 Breadth first and uniform cost search

BFS FIFO queue. Complete (finite b). Fewest actions, optimal only for equal costs. Time and space $O(b^d)$.

UCS Priority queue on g(n). Complete and optimal if every cost ≥ ε > 0. $O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$.

Goal test timing BFS: test at generation. UCS: test at expansion (Sibiu: 310 is replaced by 278).

Romania BFS 450 via Fagaras. UCS 418 via Rimnicu Vilcea and Pitesti, after taking out 13 cities.

Optimality proof Lemma 1: costs taken out never decrease. Lemma 2: cheaper paths come first. Lemma 3: first path to a state is cheapest.

Memory b = 10, d = 10 needs about 10 TB. Memory runs out before time.

Time of BFS $1 + b + b^2 + \ldots + b^d = O(b^d)$

Space of BFS
$O(b^{d-1})$ explored + $O(b^d)$ frontier = $O(b^d)$

Time and space of UCS $O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$

Geometric series
$\sum_{i=0}^{d} b^i = (b^{d+1} - 1)/(b - 1) \le b^d \cdot b/(b - 1)$

TRAP Two lessons from the table. Memory runs out long before time does. And no amount of faster hardware saves you: each extra level multiplies the cost by b.

EXAM This is why UCS tests the goal when a path is taken OUT, not when it is generated. In the Sibiu example, Bucharest was generated with cost 310 before the 278 path existed. Testing at generation would have returned the wrong answer.

2.2 Depth first, iterative deepening and bidirectional search

DFS Stack. O(b m) space, $O(b^m)$ time. Not complete on infinite or cyclic spaces, not optimal.

DLS DFS with depth limit D. Returns solution, failure, or cutoff. Complete only if d ≤ D.

IDS DLS with D = 0, 1, 2, ... Complete, optimal for equal costs, $O(b^d)$ time, O(b d) space.

IDS overhead $\sum (d + 1 - i)\, b^i$, about $b/(b - 1)$ times BFS. b = 10, d = 5: 123,456 against 111,111.

Path check Drop a child already on the current path. Removes cycles, keeps O(b m) memory.

Bidirectional Search from both ends, $O(b^{d/2})$. Needs reversible actions and one explicit goal.

Nodes generated by IDS
$N(IDS) = (d + 1)\cdot 1 + d\cdot b + (d - 1)\cdot b^2 + \ldots + 1\cdot b^d = \sum_{i=0}^{d} (d + 1 - i)\, b^i$

Overhead of IDS over BFS
$N(IDS)/N(BFS) \approx b/(b - 1)$

Cost of bidirectional search
$2 \cdot b^{d/2} = O(b^{d/2})$ instead of $O(b^d)$

EXAM A path check makes DFS complete on any finite state space, while keeping memory at O(b m). It can still visit the same state many times along different paths, so the time may be much more than the number of states.

TRAP When the frontiers first meet, the joined path is not always the shortest. With uniform costs, finish the current level before stopping, then pick the best meeting point.

STEP 3 Core AI · Informed search

3.1 Greedy best first and A* search

Greedy best first Order by h. Not optimal (Romania 450, slide 58 trap 110). Can loop in tree search.

A* Order by f = g + h. Romania: Arad 366, Sibiu 393, RV 413, Fagaras 415, Pitesti 417, Bucharest 418.

Admissible h(n) ≤ h*(n). Makes A* tree search optimal.

Consistent h(n1) ≤ C(n1, a, n2) + h(n2). f never decreases, graph search optimal, implies admissible.

Optimally efficient A* expands every node with f < C* and none with f > C*. No optimal method with the same h does better.

Weighted A* f = g + w h, cost ≤ w C*.

Evaluation function of A* $f(n) = g(n) + h(n)$

Admissible $h(n) \le h^*(n)$ for every n

Consistent (monotone)
$h(n1) \le C(n1, a, n2) + h(n2)$ for every action a from n1 to n2

Optimally efficient
no other optimal algorithm using the same h expands fewer nodes (up to ties)

Bounded suboptimality cost found ≤ w · C*

EXAM Food for thought IV asks for necessary and sufficient conditions. For optimal A*: admissibility is sufficient for tree search, consistency is sufficient for graph search. Neither is necessary: a slightly overestimating h can still return the optimal path by luck, but then nothing is guaranteed.

TRAP The time of A* is still exponential in general, and the memory is its real weak point: it keeps every generated node. That is why chapter 3.3 builds IDA*, RBFS and SMA*.

3.2 Designing and comparing heuristics

8 puzzle heuristics h1 misplaced tiles, h2 Manhattan distance. Textbook start: h1 = 8, h2 = 18, true cost 26.

Relaxed problem Remove rules. The exact relaxed cost is admissible and consistent.

Dominance h2 ≥ h1 and both admissible: A* with h2 expands no more nodes.

Combining max of admissible heuristics is admissible and dominates each one.

Effective branching factor $N + 1 = 1 + b^* + \ldots + (b^*)^d$. Close to 1 is good.

Pattern database Exact costs of a sub problem, looked up during search. Disjoint PDBs can be added.

h2: Manhattan distance
$h2(n) = \sum_{\text{tiles}} (|\text{row} - \text{goal row}| + |\text{column} - \text{goal column}|)$

Dominance
h2 dominates h1 if $h2(n) \geq h1(n)$ for every n, and both are admissible

Combining $h(n) = \max(h1(n), \ldots, hk(n))$

Disjoint pattern databases
$h = \text{PDB(tiles 1 to 4)} + \text{PDB(tiles 5 to 8)}$

Effective branching factor b*
$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$

TRAP Larger is better only while h stays admissible. 2 × h2 is larger, but it overestimates, so A* may then return a longer solution.

3.3 Memory bounded heuristic search: IDA*, RBFS and SMA*

Why A* stores every node and runs out of memory first. Memory bounded methods trade memory for repeated work.

IDA* DFS with an f cutoff, next cutoff = smallest f over the old one. Romania: 366, 393, 413, 415, 417, 418.

RBFS f-limit = min(limit, best alternative). Unwind when best f > limit and back up that f. Linear memory.

SMA* A* until memory is full, then drop the highest f leaf and back up its f. Optimal if the solution fits.

Costs IDA* and RBFS: O(b d) memory. IDA* worst case $O(N^2)$ time with many distinct f values.

Iterations of IDA*
one per distinct f value up to C*

Memory
A*: $O(b^d)$ IDA*, RBFS: O(b d) SMA*: M, the memory you give it

EXAM The minor exam of this course was announced as a coding task, such as writing RBFS. The code in lesson 3 is short enough to learn by heart, and the notebook lets you practise it.

3.4 AND OR search and AO*

AND and OR nodes OR: the agent chooses, one child must work. AND: nature chooses, every child must work.

Contingency plan Erratic vacuum from state 1: [Suck, if State = 5 then [Right, Suck] else []].

AND OR search DFS that fails a branch on a state already on its path. Complete on finite spaces.

Cyclic plan Slippery world: [Suck, L1: Right, if State = 5 then L1 else Suck]. Needs fairness.

AO* Expand the best partial graph, AND = sum, OR = min, revise upward. Slide example: 6, then 9 against 8, ends at 5. Optimal only with admissible h.

A cyclic plan
[Suck, L1: Right, if State = 5 then L1 else Suck]

Cost of an AND connector
$\text{cost}(n, k) = \sum_{\text{children c of k}} (C(n, c) + \text{cost}(c))$

TRAP A cyclic plan only works under a FAIRNESS assumption: an action that can succeed will succeed if you try it often enough. If the failure has a hidden cause (the wheel is broken), trying again is useless and the agent must learn the cause instead.

TRAP Look closely at the slide numbers: h(D) = 4 but D really costs only 1 (D → J). So this h overestimates and is not admissible. Here AO* still ends at the best answer 5, but try the second tip (h(H) = 6): AO* stops at 9 while the true best is 7. Like A*, AO* is only guaranteed optimal when h never overestimates.

STEP 4

Core AI · Local and evolutionary search

4.1 Local search: hill climbing and simulated annealing

Local search One current state, an objective, neighbours. No path, constant memory, works on huge spaces.

Hill climbing Best neighbour until none is better. Stuck at local maxima, ridges and plateaux. 8 queens: about 14 percent solved.

Fixes Sideways moves (about 94 percent), random restarts (1/p runs), stochastic and first choice, local beam search.

Simulated annealing Accept worse with $e^{\Delta E/T}$. High T explores, low T exploits.

Cooling schedules Geometric $T \leftarrow \alpha T$ ($0.8 \leq \alpha \leq 1$), linear $T0 - k\Delta T$, logarithmic $C/\ln(k + 2)$, constant, exponential $T0\ \alpha^k$.

Random restart expected restarts = $1/p$

Acceptance probability
$P(\text{accept}) = e^{\Delta E/T}$ for $\Delta E < 0$, and 1 for $\Delta E \geq 0$

Gradient ascent for continuous states
$x \leftarrow x + \eta \nabla f(x)$

4.2 Genetic algorithms

GA words Gene, chromosome, population, fitness, selection, crossover, mutation.

Roulette wheel $P(i) = f_i / \sum f$. Expected copies $f_i / \bar{f}$.

Slide example Fitness 169, 576, 64, 361, total 1170, average 292.5. Children 01100, 11001, 11011 (729), 10000.

Other selections Tournament, rank, elitism. Rank and tournament ignore the scale of fitness.

When GAs help When good solutions are built from blocks crossover can combine. Watch diversity to avoid premature convergence.

Roulette wheel probability $P(i) = f_i / \sum_j f_j$

TRAP The slide table lists x = 10 for the string 10000. In binary 10000 is 16, and $16^2 = 256$ matches the f column, so x should read 16.

TRAP For problems like the travelling salesperson, plain one point crossover makes invalid tours (a city twice). Use order crossover or swap mutation, which always give valid permutations.

STEP 5

Core AI · Constraint satisfaction

5.1 Constraint satisfaction problems and arc consistency

CSP Variables X, domains D, constraints C. Solution: complete and consistent assignment.

Examples Australia: $3^7 = 2{,}187$ assignments, 18 solutions. N queens with $X_{ij} \in \{0, 1\}$. Car assembly with precedence T1 + d1 ≤ T2 and disjunctive constraints.

Arc consistency $\forall x \in \text{dom}(X)\ \exists y \in \text{dom}(Y)$: r(x, y). AC-3 costs $O(c\ d^3)$.

Three outcomes Empty domain: no solution. All singletons: unique solution. Otherwise search.

Forward checking against AC-3 FC checks arcs from the newly assigned variable only. AC-3 checks every arc and finds failures earlier.

Arc consistency
$\forall x \in \text{dom}(X)\ \exists y \in \text{dom}(Y)$: r(x, y)

Cost of AC-3 $O(c\ d^3)$

Number of complete assignments d^n

Naive search tree $n! \cdot d^n$ leaves

TRAP Arc consistency cannot see that 2 colours are not enough for a triangle WA, NT, SA: each arc on its own is fine ({red, blue} against {red, blue}). Path consistency catches it.

5.2 Backtracking search for CSPs

Backtracking DFS, one variable per level (assignments commute), undo on a broken constraint. d^n leaves.

MRV and degree MRV: fewest legal values (fail first). Degree: most constraints on unassigned variables, breaks ties. SA first.

LCV Value ruling out fewest neighbour values first. WA = red, NT = green: Q = red, not blue.

FC and MAC FC prunes neighbours of the assigned variable. MAC runs AC-3 and catches more.

Backjumping and learning Jump to the latest variable in the conflict set. Record no goods so the same failure never repeats.

Worst case of backtracking $O(d^n)$

EXAM Variable ordering is fail first: pick the hardest variable so a dead end shows up early. Value ordering is succeed first: pick the value most likely to work. When you want all solutions, or none exists, value ordering does not matter, because every value gets tried anyway.

TRAP Forward checking misses conflicts between two unassigned variables. After WA = red and Q = green, NT and SA both have only blue left and are neighbours. Forward checking does not see it. MAC does, because it also checks the arc NT, SA.

5.3 Local search for CSPs and problem structure

Min-conflicts Complete random assignment, move a random conflicted variable to its least conflicting value. About 50 steps for a million queens (AIMA).

Escaping traps Sideways moves, tabu search, constraint weighting, restarts. Local search cannot prove no solution.

Tree CSP O(n d^2): directional arc consistency backward, then assign forward, no backtracking.

Cutset conditioning O(d^c(n − c)d^2). Australia: cutset {SA}.

Tree decomposition O(n d^{w+1}), w = tree width.

Independent sub problems
connected components solved separately: $O(d^c \cdot n/c)$ instead of $O(d^n)$

Tree structured CSP solvable in $O(n\ d^2)$

Cutset conditioning $O(d^c \cdot (n - c)\ d^2)$

TRAP Local search cannot prove that a CSP has no solution. It just fails to find one. Backtracking with propagation can prove it, for example that 2 colours are not enough for Australia.

STEP 6 Core AI · Adversarial search

6.1 Games and minimax

Game S_0, PLAYER, ACTIONS, RESULT, TERMINAL-TEST, UTILITY. Zero sum: one player's gain is the other's loss.

Minimax MAX takes max, MIN takes min, terminals give utility. Root value = what MAX can guarantee.

Slide examples 27 leaves: B1 = 5, B2 = 2, B3 = 3, A = 5. Tic-tac-toe: O takes the centre.

Cost Complete (finite), optimal against perfect play, O(b^m) time, O(b m) space. Chess ≈ 10^{154}.

Cutoffs Depth limit with EVAL = $\sum w_i f_i$. Quiescence search, horizon effect.

Minimax value
MINIMAX(s) = UTILITY(s) if s is terminal, $\max_a$ MINIMAX(RESULT(s, a)) if MAX moves, $\min_a$ MINIMAX(RESULT(s, a)) if MIN moves

Depth limited minimax
replace TERMINAL-TEST by CUTOFF-TEST(s, d) and UTILITY by EVAL(s)

Zero sum
UTILITY(s, MAX) + UTILITY(s, MIN) = constant

6.2 Alpha beta pruning

α and β α = best MAX can guarantee so far (starts −∞). β = best MIN can guarantee so far (starts +∞).

Rule α, β go parent to child. Values go child to parent. Prune when α ≥ β. Root value = minimax.

Slide example Leaves 2 3 5 9 0 1 7 5. Prune the 9 under E and all of G. A = 3 after 5 of 8 leaves.

Cost Best O($b^{m/2}$), random about O($b^{3m/4}$), worst O(b^m). Space O(b m). Same time, twice the depth.

Ordering Best first: $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$ leaves. Iterative deepening, killer moves, transposition tables.

Fewest leaves alpha beta can read (Knuth and Moore)
$b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$

Best order cost $O(b^{m/2}) = O((\sqrt{b})^m)$

TRAP The slide text says "the initial value of alpha is +∞" in the line about beta. That is a typing slip: it is β that starts at +∞, and α starts at −∞.

EXAM A pruned node gets a bound, not an exact value: E is "at least 5" and C is "at most 1". These bounds are enough to choose the move, which is all we need at the root.

6.3 Games with chance: expectimax and Monte Carlo tree search

Chance node V(s) = ∑ p(s' | s) V(s'). exp-value starts at v = 0 and adds p × value.

Slide 11 3 12 9, 2 4 6, 15 6 0 with 1/3 each: 8, 4, 7. MAX = 8.

Slide 12 MIN 2, 1, 3 with 0.5, 0.4, 0.1: 1.7. Layers MAX, CHANCE, MIN, CHANCE.

Problems Need probabilities, cost O(b^m n^m), value scale matters (only a x + b with a > 0 is safe), pruning needs bounds.

MCTS Select by UCB1 = w/n + C √(ln N/n), expand, simulate, back up. Play the most visited move.

Expectimax value (slide 11)
$V(s) = \max_{s' \in \text{successors(s)}} V(s')$ for agent controlled states. $V(s) = \sum_{s' \in \text{successors(s)}} p(s' \mid s)\ V(s')$ for chance states. V(s) = known for terminal states

Expectiminimax
EMM(s) = UTILITY(s) if terminal, $\max_a$ EMM(RESULT(s, a)) if MAX, $\min_a$ EMM(RESULT(s, a)) if MIN, $\sum_r P(r)$ EMM(RESULT(s, r)) if CHANCE

UCB1 (upper confidence bound)
$UCB1(i) = w_i/n_i + C\sqrt{(\ln N/n_i)}$

Expected value $E[X] = \sum_x P(x) \times x$

STEP 7 Core AI · Logic and knowledge

7.1 Knowledge, logic and propositional logic

KB agent TELL percepts, ASK for an action, TELL the action. Inference happens inside ASK.

Entailment KB ⊨ α iff M(KB) ⊆ M(α). Sound: derives only entailed. Complete: derives all entailed.

Connectives ¬ ∧ ∨ ⇒ ⇔, in that order of precedence. P ⇒ Q false only for P = T, Q = F.

Wumpus KB R1 ¬P11, R2 B11 ⇔ (P12 ∨ P21), R3 B21 ⇔ (P11 ∨ P22 ∨ P31), R4 ¬B11, R5 B21. 3 of 128 models. Entails ¬P12.

Validity KB ⊨ α iff KB ⇒ α is valid iff KB ∧ ¬α is unsatisfiable. SAT is NP-complete.

Entailment
KB ⊨ α if and only if M(KB) ⊆ M(α)

Deduction theorem
KB ⊨ α if and only if (KB ⇒ α) is valid

Proof by contradiction (refutation)
KB ⊨ α if and only if (KB ∧ ¬α) is unsatisfiable

Number of models 2^n for n symbols

TRAP P ⇒ Q is true whenever P is false. "If 5 is even, then Sam is smart" is true in propositional logic, whatever Sam is like. Implication does not mean that P causes Q.

7.2 Reasoning patterns: rules, resolution and chaining

Rules Modus Ponens: α ⇒ β, α ⊢ β. And-Elimination. Equivalences as rules. Monotonic.

Resolution ℓ ∨ α, ¬ℓ ∨ β ⊢ α ∨ β. One pair at a time. Refutation complete.

CNF steps Remove ⇔, remove ⇒, move ¬ in (De Morgan, double negation), distribute ∨ over ∧.

Chaining Definite clause: exactly one positive literal. Forward from facts, backward from goal. Linear time.

SAT DPLL: early termination, pure symbol, unit clause. WalkSAT: local search, incomplete. CDCL in practice.

The resolution rule
from $\ell_1 \vee \ldots \vee \ell_k$ and $m_1 \vee \ldots \vee m_n$, where ℓ_i and m_j are opposite literals, conclude the clause of all the other literals

Possible clauses over n symbols 3^n

EXAM The ground resolution theorem: if a set of clauses is unsatisfiable, resolution will reach the empty clause. So resolution is a complete refutation procedure for propositional logic. It cannot list every entailed sentence directly, but it can always confirm or refute a given one.

7.3 First order logic: syntax and semantics

Syntax Constants, predicates, functions, variables. Terms name objects, atoms are true or false.

Semantics Model = domain + interpretation. Infinitely many models, so no truth tables.

Quantifier traps ∀ with ⇒, ∃ with ∧. ∀x ∃y ≠ ∃y ∀x. ¬∀x P ≡ ∃x ¬P.

Equality x = y means same object. Needed for "at least two". Database semantics: unique names, closed world, domain closure.

Engineering Questions, knowledge, vocabulary, general axioms, instance facts, queries, debug.

De Morgan for quantifiers
∀x ¬P ≡ ¬∃x P, ¬∀x P ≡ ∃x ¬P, ∀x P ≡ ¬∃x ¬P, ∃x P ≡ ¬∀x ¬P

Interpretations over a fixed domain of d objects
$d^c \times (2^d)^u \times (2^{d^2})^b$

TRAP First order logic has infinitely many models: any domain size from 1 upwards, and every interpretation of every symbol. So entailment cannot be checked by listing models, as we did with truth tables.

7.4 Inference in first order logic: unification, chaining and resolution

UI and EI UI: any ground term, many times. EI: new Skolem constant, once.

Propositionalisation: k^V copies. Semidecidable.

Unify {x/Jane}, {y/John, x/Bill}, {y/John, x/Mother(John)}, fail for Knows(x, Elizabeth). Occurs check. Standardise apart. MGU.

GMP and FC Generalised Modus Ponens with θ. Crime: round 1 Weapon, Sells, Hostile, round 2 Criminal(West). Datalog stops.

BC Goal driven, depth first, left to right (Prolog). Left recursion loops. Negation as failure.

FOL resolution CNF: remove ⇒, ¬ in, standardise, Skolemise (∃y inside ∀x gives F(x)), drop ∀, distribute. Refutation complete.

Generalised Modus Ponens (GMP)
from $p_1', ..., p_n'$ and $(p_1 \wedge ... \wedge p_n \Rightarrow q)$, infer $SUBST(\theta, q)$, where $SUBST(\theta, p_i') = SUBST(\theta, p_i)$ for every i

Binary resolution
from $\ell_1 \vee ... \vee \ell_k$ and $m_1 \vee ... \vee m_n$ with $UNIFY(\ell_i, \neg m_j) = \theta$, infer $SUBST(\theta$, all other literals)

Ground instances of one rule k^V

TRAP The occurs check (does the variable appear inside the term?) makes unification cost quadratic in the size of the terms. Prolog leaves it out for speed, so Prolog can unify x with F(x) and build an endless term.

STEP 8 Core AI · Planning

8.1 Situation calculus and deductive planning

Situations S0, Result(a, s). Fluents take s as last argument. Atemporal predicates do not.

Axioms Possibility: Poss(a, s) ⇔ preconditions. Effect: Poss(a, s) ⇒ effect in Result(a, s).

Frame problem Effect axioms do not say what stays. A × F frame axioms, or F successor state axioms (Reiter).

SSA form F(Result(a, s)) ⇔ a made F true ∨ (F(s) ∧ a did not make F false). Needs unique action axioms.

Deductive planning Green 1969: prove ∃s Goal(s), extract s. Slow, so STRIPS (1971).

Successor state axiom (one per fluent F)
$Poss(a, s) \Rightarrow [F(Result(a, s)) \Leftrightarrow$ (a made F true) ∨ (F(s) ∧ a did not make F false)]

Frame axioms needed $A \times F$

Successor state axioms needed F

8.2 STRIPS planning and sub goals

STRIPS State = set of facts. Action = PRE, ADD, DEL. RESULT = (s − DEL) ∪ ADD. Unmentioned facts stay.

Regression g' = (g − ADD) ∪ PRE, for actions that add part of g and delete none of it.

Heuristics Ignore preconditions, ignore delete lists (h^+ admissible, h^{FF} greedy), sub goal independence.

Sussman anomaly C on A, goal A on B and B on C. Either sub goal order undoes the other. Shortest plan 3: C to table, B on C, A on B.

Complexity STRIPS planning is PSPACE-complete. Blocks world states: 13 for 3 blocks, 73 for 4.

Applying an action
$RESULT(s, a) = (s - DEL(a)) \cup ADD(a)$, if $PRE(a) \subseteq s$

Regression $g' = (g - ADD(a)) \cup PRE(a)$

Ground actions in the blocks world with n blocks
$n(n-1)$ MoveToTable + $n(n-1)$ Move from the table + $n(n-1)(n-2)$ Move between blocks

TRAP Regressing On(A, B) ∧ On(B, C) through Move(B, Table, C) gives a goal with both On(A, B) and Clear(B), which no state can satisfy. Backward planners need extra rules (state constraints) to throw such goals away early.

8.3 Partial order planning

Partial plan Steps, orderings A ≺ B, causal links A →p→ B, open preconditions. Start gives the initial state, Finish needs the goal.

Threat C deletes p and could run between A and B of the link A →p→ B.

Fixes Demotion: C ≺ A. Promotion: B ≺ C. Keep the orderings free of cycles, else go back.

Solution No open preconditions, no threats, no cycles. Every linearization then reaches the goal.

Counting Independent chains $n_1, ..., n_k$ give $(\Sigma n_i)! \div \Pi n_i!$ linearizations. Socks and shoes: 6.

Sussman by POP MoveToTable(C, A) ≺ Move(B, Table, C) ≺ Move(A, Table, B), found by two promotions.

Threat
C deletes p, and neither C ≺ A nor B ≺ C follows from the orderings

Linearizations of independent chains
$(n_1 + n_2 + ... + n_k)! \div (n_1!\, n_2! \, ... \, n_k!)$

STEP 9 Core AI · Probability and Bayesian networks

9.1 Probability for AI agents

Axioms $0 \le P(\omega) \le 1$, $\Sigma P(\omega) = 1$. $P(a \vee b) = P(a) + P(b) - P(a \wedge b)$. $P(a \mid b) = P(a \wedge b) \div P(b)$.

Enumeration $P(X \mid e) = \alpha \Sigma_y P(X, e, y)$. Marginalise, keep matching worlds, normalise. Full joint: 2^n entries.

Bayes' rule $P(c \mid e) = P(e \mid c)\, P(c) \div P(e)$, $P(e) = \Sigma_c P(e \mid c)\, P(c)$. Odds form: posterior odds = prior odds × LR.

Base rate Prior 0.01, sensitivity 0.9, false positive 0.09 gives P(ill | +) ≈ 0.092.

Conditional independence P(a, b | c) = P(a | c) P(b | c). Naive Bayes: $P(C \mid e) = \alpha P(C) \Pi P(e_i \mid C)$, 2n + 1 numbers.

Dentist numbers P(cavity) = 0.2, P(toothache) = 0.2, P(cavity | toothache) = 0.6, P(cavity | toothache, catch) ≈ 0.871.

Kolmogorov axioms
$0 \le P(\omega) \le 1$ for every world ω, and the sum of P(ω) over all worlds is 1

Conditional probability
$P(a \mid b) = P(a \wedge b) \div P(b)$, when $P(b) > 0$

Inference by enumeration
$P(X \mid e) = \alpha\, P(X, e) = \alpha \Sigma$ over y of $P(X, e, y)$

Bayes' rule $P(c \mid e) = P(e \mid c)\, P(c) \div P(e)$

Conditional independence
$P(a \wedge b \mid c) = P(a \mid c)\, P(b \mid c)$

Chain rule
$P(x_1, ..., x_n) = \Pi$ over i of $P(x_i \mid x_1, ..., x_{i-1})$

TRAP P(a | b) is not "if b then a with probability p". It is belief in a when b is all the agent knows. If more evidence c arrives, the agent must use P(a | b ∧ c).

TRAP The base rate fallacy: most people guess about 90 percent for the test above. Healthy people are so many that their few false positives outnumber the true positives.

9.2 Bayesian networks and Markov networks

Bayesian network DAG + CPT per node. $P(x_1..x_n) = \Pi P(x_i \mid \text{parents})$. Size Σ 2^(parents). Burglary: 10 vs 31.

d-separation Chain and fork blocked if the middle is observed. Collider blocked unless the middle or a descendant is observed.

Markov blanket Parents, children, children's other parents. Given it, a node is independent of all others.

Noisy OR $P(\neg x \mid \text{causes}) = \Pi$ q_j over present causes. n numbers instead of 2^n.

Markov network $P(x) = (1 \div Z) \Pi \varphi_c(x_c)$, $Z = \Sigma_x \Pi \varphi_c$. Separation in the graph gives independence. Moralise a BN: marry parents, drop arrows.

Key numbers P(j, m, a, ¬b, ¬e) ≈ 0.000628. Misconception network Z = 7,201,840.

Network semantics
$P(x_1, ..., x_n) = \Pi$ over i of $P(x_i \mid \text{parents}(X_i))$

Markov blanket
parents, children, and the children's other parents

Noisy OR
$P(\neg x \mid \text{parents}) = \Pi$ over true parents j of q_j

Gibbs distribution
$P(x) = (1 \div Z) \Pi$ over cliques c of $\varphi_c(x_c)$

Parameters with at most k parents
$n\, 2^k$, against $2^n - 1$ for the full joint

9.3 Inference in Bayesian networks

Enumeration $P(X \mid e) = \alpha \Sigma_y \Pi P(x_i \mid \text{parents})$. $O(2^n)$ time, O(n) space. P(b | j, m) ≈ 0.284.

Variable elimination Factors from CPTs with evidence. Multiply factors with X, sum out X. Cost exponential in the largest factor (treewidth). Drop non ancestors.

Complexity Polytree: linear. General: #P-hard (Cooper 1990).

Samplers Rejection: N(x, e) ÷ N(e). LW: $w = \Pi P(e_i \mid \text{parents})$. Gibbs: sample from P(x | mb(X)). Error ≈ √(p(1 − p) ÷ N).

Key numbers LW sample weight 0.1 × 0.99 = 0.099. P(Rain | s, w) = 0.3204. P(Rain | s) = 0.3.

Enumeration
$P(B \mid j, m) = \alpha \Sigma_e \Sigma_a P(B)\, P(e)\, P(a \mid B, e)\, P(j \mid a)\, P(m \mid a)$

Summing out $h(X, Z) = \Sigma_y f(X, y, Z)$

Likelihood weight
$w = \Pi$ over evidence variables E_i of $P(e_i \mid \text{parents}(E_i))$

Likelihood weighting is consistent
$S_WS(z, e)\ w(z, e) = \Pi\ P(z_i \mid \text{parents}) \times \Pi\ P(e_i \mid \text{parents}) = P(z, e)$

TRAP Enumeration repeats work: P(j | a) P(m | a) is computed again for each value of e. Variable elimination stores such parts once.

TRAP Likelihood weighting suffers when evidence comes late in the order (at the leaves): the samples ignore it while choosing the earlier variables, so most of them get tiny weights.

9.4 Causality: seeing, doing and imagining

Ladder Seeing P(y | x), doing P(y | do(x)), imagining counterfactuals.

do operator Cut arrows into X, drop its CPT (truncated product). Root X: do equals see.

Back door Z has no descendant of T and blocks all T ← ... paths: P(r | do(t)) = Σ_z P(r | t, z) P(z).

Simpson's paradox Kidney stones: seeing A 0.78 < B 0.826, doing A 0.833 > B 0.779. Adjust for confounders, not mediators or colliders.

Counterfactual Abduction (find U), action (set X), prediction (recompute Y). Front door: Σ_m P(m | x) Σ_x' P(y | m, x') P(x').

Sprinkler P(Rain | s) = 0.3 but P(Rain | do(s)) = 0.5. P(WetGrass | do(s)) = 0.945.

Truncated factorisation
$P(v \mid do(x)) = \Pi$ over i with V_i not X of $P(v_i \mid \text{parents}(V_i))$, with $X = x$

Back door adjustment
$P(r \mid do(t)) = \Sigma_z\ P(r \mid t, z)\ P(z)$

Back door $P(y \mid do(x)) = \Sigma_z\ P(y \mid x, z)\ P(z)$

TRAP A plain Bayesian network can be read in a non causal order and still give the right P(y | x). It can only answer rung 2 questions when its arrows are causal.

TRAP Simpson's paradox is not a fault in arithmetic. Which answer is right, the pooled one or the one inside groups, depends on the causal graph. With stone size as a confounder, the group answer (A is better) is right. If Z were a mediator, the pooled answer would be right.

STEP 10

Core AI · Reinforcement learning

10.1 Markov decision processes

MDP States, actions, P(s' | s, a), R, γ. Solution: policy π(s). Utility = $E[\Sigma\ \gamma^t\ R_t] \le R_max \div (1 - \gamma)$.

Bellman U(s) = R(s) + γ max_a Σ P(s' | s, a) U(s'). Q(s, a) = Σ P [R + γ max Q(s', a')]. U = max_a Q.

Value iteration Repeat the update. Contraction by γ. Stop when $\delta < \varepsilon(1 - \gamma) \div \gamma$. 4 × 3: U(1,1) = 0.705.

Policy iteration Evaluate (linear equations, $O(n^3)$), improve greedily, stop when unchanged. Few rounds.

Racing car γ = 0.9: V_1 = (2, 1), V_2 = (3.35, 2.35). Optimal: cool fast, warm slow, V = (15.5, 14.5).

Utility of a reward sequence
$U = R_0 + \gamma\ R_1 + \gamma^2\ R_2 + \ldots = \Sigma\ \gamma^t\ R_t$

Bellman equation
$U(s) = R(s) + \gamma$ max over a of Σ over s′ of $P(s' \mid s, a)\ U(s')$

Policy evaluation
$U^{\pi}(s) = R(s) + \gamma\ \Sigma\ P(s' \mid s, \pi(s))\ U^{\pi}(s')$

Bound on utilities $|U(s)| \le R_max \div (1 - \gamma)$

10.2 Passive reinforcement learning

Passive RL Fixed π, unknown P and R, learn U^{π} from trials. Direct estimation, ADP, TD.

TD(0) U(s) ← U(s) + α (R(s) + γ U(s') − U(s)). δ = TD error. Robbins Monro: Σα = ∞, Σα² < ∞.

ADP $\hat{P}(s' \mid s, a) = N(s, a, s') \div N(s, a)$, then policy evaluation. Fewest trials, most work per step.

Bias and variance Monte Carlo return: unbiased, high variance. TD target: biased, low variance. TD(λ) mixes.

Key numbers Three trials: U(1,1) ≈ 0.0933 (true 0.705). $\hat{P}((2,3) \mid (1,3)) = 2/3$. Random walk values $k \div 6$.

TD(0) update
$U(s) \leftarrow U(s) + \alpha\ (R(s) + \gamma\ U(s') - U(s))$

Learned model
$\hat{P}(s' \mid s, a) = N(s, a, s') \div N(s, a)$

TD target $R_t + \gamma\ U(S_{t+1})$

10.3 Active reinforcement learning

Explore or exploit ε-greedy, exploration function f(u, n) = R^+ if n < N_e else u, UCB1 $\bar{x} + \sqrt{(2 \ln n \div n_i)}$. GLIE.

Q-learning Q ← Q + α [r + γ max Q(s', a') − Q]. Off policy, model free. Converges with every pair tried and Robbins Monro α.

SARSA Q ← Q + α [r + γ Q(s', a') − Q] with the a' actually taken. On policy. Safer path on the cliff.

Approximation θ ← θ + α [target − $\hat{Q}$] $\nabla\hat{Q}$. DQN: replay buffer + target network. Deadly triad.

Key numbers ε-greedy best action: 1 − ε + ε/|A|. Corridor Q*: 1, 0.9, 0.81. Cliff: Q-learning path 13 steps.

Exploration function (AIMA)
$f(u, n) = R^+$ if $n < N_e$, else u

Q-learning update
$Q(s, a) \leftarrow Q(s, a) + \alpha\ [r + \gamma$ max over a′ of $Q(s', a') - Q(s, a)]$

Gradient TD update
$\theta_i \leftarrow \theta_i + \alpha\ [\text{target} - \hat{Q}(s, a)]\ \partial\hat{Q} \div \partial\theta_i$

Bellman optimality for Q
$Q^*(s, a) = \Sigma\ P(s' \mid s, a)\ [R(s, a, s') + \gamma$ max over a′ of $Q^*(s', a')]$

TRAP With function approximation, Q-learning can diverge: off policy data, bootstrapping and approximation together (the deadly triad). DQN's replay buffer and target network were added to keep training stable.

10.4 Policy search and policy gradients

Policy search Parameterise π_θ, maximise $\rho(\theta) = E[\Sigma\ \gamma^t\ R_t]$. Use stochastic (softmax) policies so ρ is smooth.

Policy gradient $\nabla\rho = E[\nabla \log \pi_\theta(a \mid s)\ Q^{\pi}(s, a)]$. Softmax: $\partial \log \pi(a) \div \partial\theta_k = 1[k = a] - \pi(k)$.

REINFORCE θ ← θ + α (G − b) ∇ log π(a | s). Baseline unbiased since $E[\nabla \log \pi] = 0$.

Actor critic δ = r + γV(s') − V(s). Actor θ += α δ ∇ log π, critic w += β δ ∇V. PPO clips the ratio.

From experts Behaviour cloning copies actions. Inverse RL learns the reward (Ng and Russell 2000).

Policy value
$\rho(\theta) = E[\Sigma\ \gamma^t\ R_t \mid \text{actions from } \pi_\theta]$

Policy gradient theorem
$\nabla\rho(\theta) = E[\nabla_\theta \log \pi_\theta(a \mid s)\ Q^{\pi}(s, a)]$

Actor critic
$\delta = r + \gamma\ V(s') - V(s)$, $\theta \leftarrow \theta + \alpha\ \delta\ \nabla \log \pi_\theta(a \mid s)$

Log derivative trick $\nabla\ \pi(a) = \pi(a)\ \nabla \log \pi(a)$

TRAP With a greedy policy (always the best action under some score) ρ(θ) jumps when the best action changes, so its gradient is zero almost everywhere. That is why policy gradient methods use stochastic policies.

STEP 11

Core AI · Challenge and revision

11.1 Challenge problems across Core AI

Five moves Name the model, write the formula, check its conditions, work step by step, say what it means.

Search traps Admissible is not enough for graph search without reopening. IDS costs about b ÷ (b − 1) times BFS.

CSPs and games Tree CSP $O(n\ d^2)$. Alpha beta best case $b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$. Expectimax needs real values.

Logic and planning Resolution refutes KB ∧ ¬α. Occurs check. Regress: (goal − add) ∪ pre. Promotion or demotion.

Uncertainty and RL Base rates. Observed descendant opens a collider. $\|BU - BU'\| \le \gamma\ \|U - U'\|$. Q += α (r + γ max Q' − Q).

EXAM The fix: use a consistent heuristic, or let graph search reopen a closed node when a cheaper path to it turns up. Tree search with an admissible heuristic is always optimal, because it never closes anything.

EXAM Forward chaining is complete for definite clauses only. Resolution is refutation complete for any clauses. A favourite exam trap gives a KB with a disjunction such as A ∨ B and asks for forward chaining.

EXAM The answer P(B | j, m) ≈ 0.284 from chapter 9.3 is worth knowing by heart. Exams often give the burglary tables and ask for it, or for P(J | b) by summing out A.

11.2 Exam revision: from school to research universities

Rows of a truth table 2^n

Nodes at depth d b^d

EXAM Round only at the very end. A numerical answer type key often has a narrow range such as 0.69 to 0.70.

EXAM In proofs, name each condition when you use it: consistency, positive step costs, definite clauses, a finite domain, γ < 1. Examiners often ask what breaks when one fails.

STEP 1

Advanced AI · Rational choice

1.1 Decision problems and preferences

Decision problem (A, X, c, ≥): actions, outcomes, consequence map c : A → X, preference. Utility goes on outcomes, v(a) = u(c(a)).

Three relations Weak x ≥ y. Strict x ≻ y means x ≥ y and not y ≥ x. Indifference x ∼ y means both directions.

Rationality Complete (every pair compared) plus transitive (rankings chain). Cycles have no best element and allow a money pump.

What carries over Transitivity of ≥ passes to ≻ and ∼. Completeness does not pass to either.

Indifference is not incompleteness Indifferent: equally good. Incomplete: cannot compare. Equality: the same object.

A decision problem $(A, X, c, \succeq)$

Weak preference $x \succeq y$

Strict preference
$x \succ y \iff x \succeq y$ and not $y \succeq x$

Indifference $x \sim y \iff x \succeq y$ and $y \succeq x$

Completeness
for all $x, y \in X$: $x \succeq y$ or $y \succeq x$

Transitivity $x \succeq y$ and $y \succeq z \implies x \succeq z$

TRAP Equality and indifference are different ideas. Equality means two things are the same object. Indifference means two different outcomes are judged equally good. Tea and coffee are not equal, but you may be indifferent between them.

TRAP Completeness does not mean you like every option. It means you can compare any two, possibly by saying you are indifferent.

1.2 Utility functions

Representation x ≥ y ⟺ u(x) ≥ u(y). Strict gives >, indifference gives =. Not "=" for weak preference.

Ordinal Only order matters. Any strictly increasing f gives f(u), another valid utility. Gaps and ratios mean nothing.

Existence Finite X + complete + transitive ⟹ a utility exists. Build it by peeling off top classes: A ≻ B ∼ C ≻ D gives 3, 2, 2, 1.

Best exists In finite X an endless chain of improvements must cycle, which transitivity forbids. Ties at the top are allowed.

Counting n strict outcomes give n! rankings (5! = 120). With ties: 1, 3, 13, 75 weak orders.

Representation $x \succeq y \iff u(x) \geq u(y)$

Ordinal invariance
if f is strictly increasing, $v(x) = f(u(x))$ represents the same ≥

Existence theorem
X finite and ≥ complete and transitive ⟹ some $u : X \to \mathbb{R}$ represents ≥

TRAP The slide transcript once read "u(x) = u(y) ⟺ x ≥ y". That is a transcription slip. The correct condition for weak preference uses ≥. Equality belongs to indifference only.

TRAP With ordinal utility, differences mean nothing. u(A) − u(B) = 2 does not say A is liked twice as much as some other pair with difference 1.

1.3 Choosing the best action

Rational choice v(a) = u(c(a)). a* ∈ arg max v, i.e. v(a*) ≥ v(a) for all a. Ties are all rational.

Movie tree Utilities 6 to 1, minus 1 per mile. INOX 1 mile, PVR 3. Best: INOX Aliens, net 5.

Fruit trees 1.6 Plain: AA at X − 60. Second same tree saves half: AA, OO become X + 5, PP X + 22.5, best AP at X − 55.

Parks 1.7 v = √(400c) − c/80 gives √c = 800, c* = 640,000 under the 1,000,000 cap. With √(1600c) the peak 2,560,000 is infeasible, so c* = 1,000,000.

Other ideas Satisficing picks the first v ≥ T. Minimizing loss = maximizing −loss. Check feasibility and boundaries.

Value of an action $v(a) = u(c(a))$

Rational choice
$a^* \in \arg\max_{a \in A} v(a) \iff v(a^*) \geq v(a)$ for every a

Spending after a plan
$u(\text{plan}) = X + \Sigma \text{ maintenance} - \Sigma \text{ reduction}$

Benefit $v(c) = \sqrt{400c} - \frac{c}{80} = 20\sqrt{c} - \frac{c}{80}$

First order condition $v'(c) = \frac{10}{\sqrt{c}} - \frac{1}{80} = 0$

TRAP The transcript sometimes wrote v(a*) > v(a) for every other a. That strict version only fits when the best action is unique. With ties the right condition is ≥.

TRAP Setting the derivative to zero is not enough. Check the second derivative and the end points of the allowed range.

STEP 2

Advanced AI · Choice under risk

2.1 Lotteries and distributions

Lottery Action a gives P(xᵢ | a) = pᵢ with pᵢ in [0, 1] and Σ pᵢ = 1. Nature picks the outcome.

CDF F(x) = P(X ≤ x | a). Non decreasing, 0 at −∞, 1 at +∞, right continuous. F(50) = 0.7 is "at most 50".

Intervals P(r < X ≤ s) = F(s) − F(r). Notes example 0.8 − 0.3 = 0.5.

Density F = ∫ f, f = F'. P(X = x) = 0 for continuous X. Densities can exceed 1.

Compound paths Multiply along each path, add the paths that reach the same outcome: 0.6 × 0.8 + 0.4 × 0.3 = 0.6.

Lottery of an action
$P(x_i \mid a) = p_i, \quad 0 \leq p_i \leq 1, \quad \sum p_i = 1$

CDF of an action $F_a(x) = P(X \leq x \mid a)$

Interval probability
$P(r < X \leq s \mid a) = F_a(s) - F_a(r)$

From density to CDF $F_a(x) = \int_{-\infty}^{x} f_a(t)\,dt$

TRAP F(50) = 0.7 means a 70% chance the outcome is at most 50. It does not mean a 70% chance the outcome is exactly 50. This misreading is the most common exam error on CDFs.

TRAP A density can be larger than 1. The uniform density on [0, 0.5] equals 2. Only the area under it must be 1.

2.2 Expected utility and compound lotteries

Expected utility EU(a) = Σ P(xᵢ | a) u(xᵢ), or ∫ u dF. Choose arg max EU, not arg max E[X].

R&D compound lottery P(10 | G) = 0.625 × 0.9 + 0.375 × 0.5 = 0.75, EU(G) = 7.5 > EU(S) = 5. Multiply along paths, add paths.

Reversal u = 9, −1, 10, 0: EU(G) 6.5 > 5. u(B) = −8: 4.75 < 5. Same ranking, opposite choice.

Affine invariance Only v = a u + b with a > 0 keeps every EU comparison. Utility under risk is cardinal.

Other rules and PV Optimistic (P best), pessimistic (P worst). PV = FV ÷ $(1 + r)^t$. vNM axioms give EU.

Expected utility, discrete
$EU(a) = \sum_i P(x_i \mid a)\, u(x_i)$

Rational choice under risk
$a^* \in \arg\max_a \mathrm{E}[\, u(X) \mid a \,]$

Reduction of compound lotteries
$P(\text{final outcome}) = \sum_{\text{paths}} \prod_{\text{branches on the path}} P(\text{branch})$

Positive affine transformation
$v(x) = a\,u(x) + b, \quad a > 0$

EXAM Moving u(B) from −1 to −8 is not affine: it stretches one gap and leaves the others alone. Under risk the sizes of the gaps matter, so expected utility needs cardinal utility, unique up to a u + b with a > 0.

TRAP Expected value and expected utility differ whenever u is not a straight line. Write which one a question asks for.

2.3 Risk attitudes and the St Petersburg paradox

Three attitudes Concave u (ln, √): averse, E[u] ≤ u(E[X]). Linear: neutral. Convex (x²): loving. Jensen with the chord picture.

Certainty equivalent u(CE) = E[u(X)]. Risk premium = E[X] − CE. √x with 100 at 0.2: EU = 2, CE = 4, sure 30 wins.

Fair gamble +1 or −1 at 50/50 has mean 0. Play when 0.5u(1) + 0.5u(−1) > u(0). Averse agents refuse.

St Petersburg Pays 2^{n-1} w.p. $1/2^n$. E[X] = Σ ½ = ∞. Log utility: E = ln 2, CE = 2. x² still diverges.

Practice 5 u = x², 0 or 10: E[X] = 5, u(E[X]) = 25, E[u] = 50, so risk loving.

Risk neutral $\mathrm{E}[\, u(X) \,] = u(\, \mathrm{E}[X] \,)$

Risk averse $\mathrm{E}[\, u(X) \,] \leq u(\, \mathrm{E}[X] \,)$

Risk loving $\mathrm{E}[\, u(X) \,] \geq u(\, \mathrm{E}[X] \,)$

Certainty equivalent
$u(\mathrm{CE}) = \mathrm{E}[\, u(X) \,] \implies \mathrm{CE} = u^{-1}(\, \mathrm{E}[u(X)] \,)$

Expected money
$\mathrm{E}[X] = \sum_{n=1}^{\infty} \frac{1}{2^n} \cdot 2^{n-1} = \sum \frac{1}{2} = \infty$

Expected log utility
$\mathrm{E}[\ln X] = \sum \frac{(n-1)\ln 2}{2^n} = \ln 2$

EXAM Concavity is what tames the paradox. With the convex u = x² the terms are 2^{n-2} and the sum still diverges. Even √x converges, to about 1.707, which gives a certainty equivalent near 2.91.

TRAP A risk averse person may still gamble. They need the gamble to pay enough more than the sure amount, the risk premium.

STEP 3

Advanced AI · Decisions over time

3.1 Backward induction

Method From the last decision back: chance node = Σ p × value, decision node = max. Never compare raw leaves at a chance node.

R&D tree Success: m 11 > d 8. Failure: m 3 < d 4. Go = 0.625 × 11 + 0.375 × 4 = 8.375 > skip 5.

Full strategies MM 8.0, MD 8.375, DM 6.125, DD 6.5. Same answer as folding, but 2^k plans for k binary nodes.

Practice A Good buzz push 14.4, weak buzz hold 3.7, big budget 7.98 > small 3.5.

Why it works Bellman's principle of optimality: each tail of an optimal plan is optimal. Folding is linear in the tree size.

Backward induction
start at the last decision, replace each chance node by its expected value, keep the best action at each decision node, and move one level back

TRAP Never compare raw leaf payoffs where nature moves. 13 is not "better than" −7 in any useful sense at a chance node, because the firm does not choose between them. Convert every chance node to its expected value first, and compare only at nodes where the firm chooses.

TRAP At chance nodes average with the probabilities. Taking the maximum there assumes luck always goes your way.

3.2 Discounting and consumption over time

Discount factor δ = 1/(1 + r), 0 < δ < 1, patience not a choice. A after n periods is worth $A\delta^n$. Practice B: PV = 520.2.

Timing Discount each piece by when it happens: R&D branch 10δ − 1, not (10 − 1)δ.

ln consumption 2 periods K/(1 + δ), δK/(1 + δ). 3 periods K, δK, δ²K over 1 + δ + δ². Equal discounted marginal utility.

Corners Linear u: (K, 0, 0). Convex u = x²: FOC finds the minimum, best is a corner. √x: x_1 = K/(1 + δ²), K = 181 gives 100, 81.

Time consistency Exponential: re-planning keeps x_2^*, x_3^*. Hyperbolic 1/(1 + kt) is time inconsistent.

Discount factor $\delta = \dfrac{1}{1+r}, \quad 0 < \delta < 1$

n periods ahead
value today of A received after n periods = $A\delta^n = \dfrac{A}{(1+r)^n}$

Two periods, u = ln x
$\max \ln x_1 + \delta \ln(K - x_1) \implies x_1^* = \dfrac{K}{1+\delta}, \quad x_2^* = \dfrac{\delta K}{1+\delta}$

Three periods, u = ln x
$x_1^* = \dfrac{K}{1+\delta+\delta^2}, \quad x_2^* = \dfrac{\delta K}{1+\delta+\delta^2}, \quad x_3^* = \dfrac{\delta^2 K}{1+\delta+\delta^2}$

TRAP The discount rate r and the discount factor δ = 1 ÷ (1 + r) are different numbers. A higher rate means a lower factor.

3.3 The value of information

VOI formula VOI = E[max_a u(a, s)] − max_a E[u(a, s)]. Choose per state then average, minus average per action then choose.

MBA Cost 10, states 0.25/0.5/0.25. No info: 9 (MBA) vs 8. Oracle: 22, 8, 4 gives 10.5. VOI = 1.5.

Never negative You can ignore the oracle, so VOI ≥ 0. It is 0 when one action is best in every state.

Venue practice C Outdoor 50/−20, indoor 30/25 at 0.5 each. 27.5 without, 37.5 with, VOI = 10. 90% forecast: 6.75.

Imperfect signals Update by Bayes, pick the best per signal, average. Worth 0 if it cannot change the decision (q = 0.5).

Value of perfect information
$\mathrm{VOI} = \mathrm{E}[\max_a u(a,s)] - \max_a \mathrm{E}[u(a,s)]$

MBA example VOI = 10.5 − 9 = 1.5

Bayes update
$P(s \mid \text{signal}) = \dfrac{P(\text{signal} \mid s)\,P(s)}{P(\text{signal})}$

TRAP A student in class computed 0.25 × 22 + 0.5 × 8 + 0.25 × 4 as the no information value. That mixes actions across states, which you can only do once you know the state. Without information you must use one action in every state.

TRAP If every forecast would lead to the same choice, the forecast is worth 0 to you, however accurate it is.

STEP 4

Advanced AI · Games between rational agents

4.1 Static games and the normal form

Two axes Complete vs incomplete information. Static (no one sees others' choices) vs dynamic. Static is about observation, not clock time.

Common knowledge Everyone knows, everyone knows everyone knows, forever. Complete info: actions, outcomes, map, preferences are common knowledge.

Normal form (N, S_i, v_i) with v_i on full profiles. Finite if all S_i finite. Bimatrix cell = (row payoff, column payoff).

Prisoner's Dilemma (M,M) = (−2,−2), (M,C) = (−5,−1), (C,M) = (−1,−5), (C,C) = (−4,−4). C is the best reply to both.

Best response $\mathrm{BR}_i(s_{-i})$ maximizes $v_i(\cdot, s_{-i})$. A profile with no profitable unilateral deviation is self enforcing.

Common knowledge
E is common knowledge $\iff$ (everyone knows)^n E for every $n = 1, 2, 3, \ldots$

Normal form game $\langle\, N, \{\{S_i\}\}, \{\{v_i\}\} \,\rangle$

Payoff function $v_i : S_1 \times \ldots \times S_n \to \mathbb{R}$

Best response
$s_i \in \mathrm{BR}_i(s_{-i}) \iff v_i(s_i, s_{-i}) \geq v_i(s'_i, s_{-i})$ for all s'_i

TRAP Payoff pairs are written (row player, column player). Swapping them is the most common slip in exam tables.

4.2 Modelling games: Cournot, Bertrand and the streetlamp

Recipe Players, strategies (with their range), payoffs split into cases. Game = (N, S_i, v_i).

Cournot P = a − bq. $s_i = (a - c_i - bs_j)/2b$, $s^* = (a - c)/3b$. a = 100, c = 10: 30 each, P = 40, profit 900.

Asymmetric Cournot $s_1^* = (a - 2c_1 + c_2)/3b$. Practice F: 18 and 12, P = 30, profits 324 and 144.

Bertrand $S_i = [0, a]$, q = (a − p)/b. Cheaper takes all, tie splits (divide by 2b), dearer gets $-C_i(0)$. Paradox: p = c.

Streetlamp $v_i = 3 \cdot 1[\Sigma \geq 2] - s_i$, 8 profiles. Pay only when pivotal. Stable: exactly two payers, or nobody.

Cournot payoff
$v_i(s_i, s_j) = (a - b(s_i + s_j))\, s_i - C_i(s_i)$, or $-C_i(s_i)$ if the price would be negative

Best response, linear cost c_i
$s_i = \dfrac{a - c_i - b\, s_j}{2b}$

Symmetric equilibrium $s^* = \dfrac{a - c}{3b}$

Bertrand payoff
$v_i = p_i \dfrac{a - p_i}{b} - C_i(\dfrac{a - p_i}{b})$ if cheaper,
half the market (divide by $2b$) if tied,
$-C_i(0)$ if dearer

Streetlamp payoff
$v_i(s) = 3 \cdot 1[\sum_j s_j \geq 2] - s_i, \quad s_i \in \{\{0, 1\}\}$

EXAM The Bertrand paradox: with equal unit cost c, the only stable prices are $p_1 = p_2 = c$. Two firms are enough to drive profit to zero, while Cournot firms keep the price at 40, well above cost 10.

TRAP The Cournot equilibrium is not the joint best outcome for the firms. Each firm ignores that its extra output lowers the other's price.

4.3 Solution concepts and Pareto optimality

Solution concept Narrows profiles to reasonable ones. Its pick is an equilibrium. Not a physics equilibrium.

Four assumptions Rational (maximize), intelligent (know the game), common knowledge of both, self enforcement (no outside enforcer).

Good concept Existence, uniqueness, invariance. "Anything can happen" exists but is not unique.

Pareto s dominates s': ≥ for all, > for one. PD: only (C, C) fails. Marketing War: (L,L), (L,M), (M,L). Zero sum: all optimal.

Altruism $v_i + ½v_j$ gives (−3,−3), (−5.5,−3.5), (−3.5,−5.5), (−6,−6). Mum becomes best. Pareto is not fairness.

Pareto dominance
s Pareto dominates $s' \iff v_i(s) \geq v_i(s')$ for all i, and > for at least one j

Pareto optimal
s' is not Pareto dominated by any profile

Altruistic payoff $v_i^{alt}(s) = v_i(s) + ½\, v_j(s)$

TRAP Pareto optimality says nothing about fairness. (2, 8) and (8, 2) are Pareto optimal next to the balanced (6, 6). It only rules out waste.

TRAP Pareto optimal and fair are different ideas. Pareto says nothing about how the total is shared.

STEP 5

Advanced AI · Dominance and iterated elimination

5.1 Dominance and the dominant strategy equilibrium

Strict domination $s_i \rhd s'_i$: $v_i(s_i, s_{-i}) > v_i(s'_i, s_{-i})$ for every s_{-i}. Row player: first numbers across columns. Column player: second numbers down rows.

Vocabulary Dominating/dominated: beats/is beaten by at least one. Dominant/submissive: beats/is beaten by all. One dominator is enough to delete.

Partial order $\rhd$ is irreflexive, asymmetric, transitive, not complete (RPS, M and H in the Marketing War). Weak: ≥ (textbook adds > once).

DSE Everyone plays a strictly dominant strategy. Unique if it exists. PD (C, C), altruistic PD (M, M), RPS and Marketing War none.

Rationality A rational player never plays a strictly dominated strategy under any belief. Increasing transforms and $h(s_{-i})$ do not change dominance.

Strict domination
$s_i \rhd_i s'_i \iff v_i(s_i, s_{-i}) > v_i(s'_i, s_{-i})$ for every s_{-i}

Strict dominant strategy equilibrium
s^D with s_i^D strictly dominant for every player i

Rational players and dominated strategies
$E_\mu[v_i(s_i)] - E_\mu[v_i(s'_i)] = \sum \mu(s_{-i})[v_i(s_i, s_{-i}) - v_i(s'_i, s_{-i})] > 0$

TRAP The mistake made twice in class: checking player 1 by mixing in the second number of each cell, or by moving down a column. Rule: for the row player compare first numbers, moving across columns. For the column player compare second numbers, moving down rows. Moving along the other player's strategies is exactly "for every s_{-i}".

TRAP Strict and weak dominance behave differently. Removing weakly dominated strategies can remove equilibria and depends on the order of removal.

5.2 Iterated elimination of dominated strategies

Algorithm Each round delete every strictly dominated strategy of every player, comparing only survivors. Stop when nothing is dominated.

Common knowledge Round k uses k layers of "I know that you know". Real people manage about 1 to 3 layers.

Slide 11 R ▷ C, then U ▷ M and D, then L ▷ R. Outcome (U, L) = (4, 3), self enforcing but Pareto dominated by (9, 6).

Theorems DSE found in one round. Every NE survives. Unique survivor = unique NE. Strict IESDS is order independent.

Limits Marketing War leaves {M, H}², RPS deletes nothing. Weak IESDS depends on order: (2, 1) or (1, 1).

IESDS, round k
$DS_i^k = \{\{ s_i \in S_i^k : \text{some } s'_i \in S_i^k \text{ beats it against every } s_{-i} \in S_{-i}^k \}\}$

Update $S_i^{k+1} = S_i^k \setminus DS_i^k$

Nash equilibrium (preview)
$v_i(s_i^*, s^*_{-i}) \geq v_i(s_i, s^*_{-i})$ for every i and every s_i

TRAP IESDS does not always leave one outcome. In many games, such as the restaurant game, nothing is dominated and every strategy survives.

5.3 Dominance at work: Cournot, auctions and voting

Cournot IESDS BR = max{0, (a − c − bq)/2b}. Rounds: [0, 45], [22.5, 45], [22.5, 33.75], ... halving to (a − c)/3b = 30. E1: 40, price 50, profit 1,600.

Second price auction Payoff v − B if b > B. Bidding v weakly dominates every bid (never worse, sometimes better). Strategy proof.

First price auction Payoff v − b if you win. Overbidding weakly dominated, but bidding v not dominant: bidders shade.

Location and prices Hotelling 1 to 5: delete 1, 5 then 2, 4, both at 3 (median). Discrete Bertrand: strict deletes nothing, weak ends at (2, 2) losing (1, 1), (3, 3).

Weak only Below cost Bertrand prices and the guess game: weakly but not strictly dominated, so strict IESDS cannot start.

Best response
$BR_i(q_j) = \max\{\{ 0, \frac{a - c - bq_j}{2b} \}\}$

Second price (Vickrey)
payoff = $v - B$ if $b > B$, otherwise 0

First price
payoff = $v - b$ if $b > B$, otherwise 0

TRAP In a second price auction you do not pay your own bid. Students who forget this conclude that shading the bid is wise.

STEP 6

Advanced AI · Quiz and exam revision

6.1 Quiz 1 solved: all four questions

First order condition $\frac{a(q_j + 1)}{(q_i + q_j + 1)^2} = c$

Symmetric equilibrium
$q^* = \frac{(a - 4c) + \sqrt{a^2 + 8ac}}{8c}$

Joint monopoly $Q^* = \sqrt{\frac{a}{2c}} - 1$

Buy one more unit when
marginal value ≥ price

Buy when
$0.4\sqrt{400 - p + 900} + 0.6\sqrt{400 - p + 10} \geq \sqrt{400} = 20$

Break even price $p^* \approx 267.99$

Answer $(x_1, x_2, x_3) = (K, 0, 0)$, $v = K^2$

EXAM The trap in S1 is pattern matching. (a − c)/3 is the textbook Cournot answer, but it comes from demand a − bq. With a different demand curve you must redo the first order condition.

TRAP This question has two true options under the standard reading (prizes added to wealth after paying p). If only one option may be marked, A is the safer choice because it covers every price below 18, and B is a special case. It is worth asking the instructor whether the question allows more than one answer.

6.2 Exam revision: from school to research universities

Expected value $E[X] = \Sigma\, p_i\, x_i$

Present value $PV = \Sigma\, \delta^t\, x_t$

EXAM Round only at the very end. A numerical answer type key often has a narrow range such as 52.6 to 52.7.

EXAM In proofs, name each assumption when you use it: completeness, transitivity, the vNM axioms, common knowledge of rationality. Examiners often ask what breaks when one fails.